

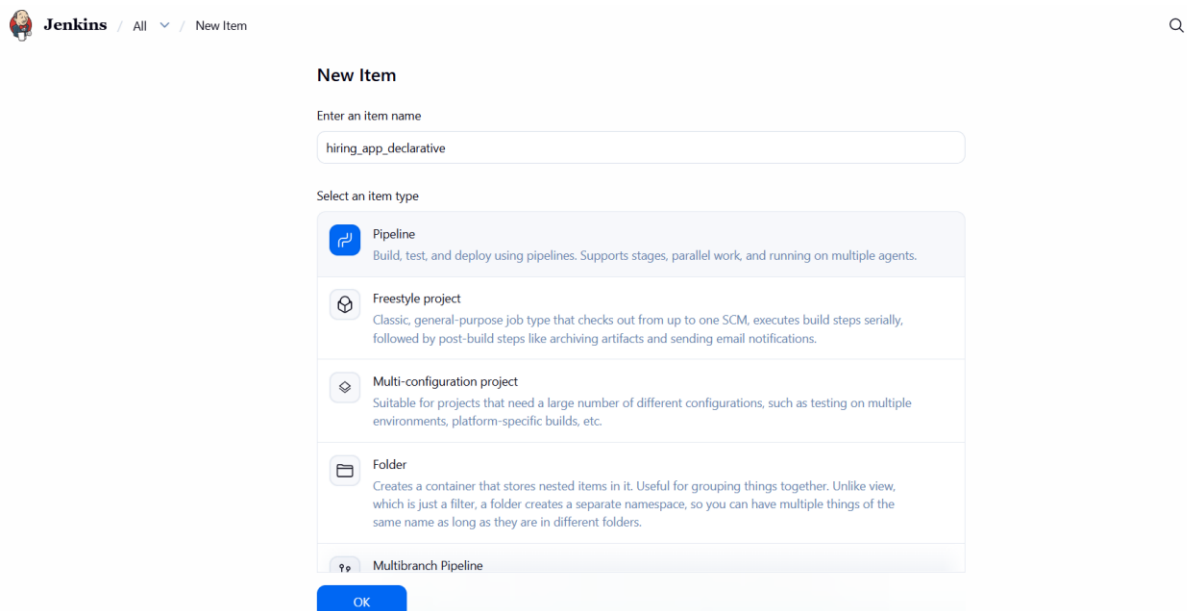
JENKINS – CHALLENGES

Use the below code and implement our CI/CD stages.

- **Source Code:** <https://github.com/betawins/hiring-app.git>
 - Repository contains the application code for the hiring app.
- 1. **Create one Declarative pipeline job**
- 2. **Create one Scripted pipeline job**
- 3. **Create one multi stage pipeline job**
- 4. **Create one parallel stage pipeline job**

1. Create one Declarative pipeline job

- Opened **Jenkins** → **New Item**
- Entered job name: **hiring_app_declarative**
- Selected **Pipeline**
- Clicked **OK**



- Opened **Configure** → **Pipeline**
- Defined:
 - agent any
 - timestamps() option

- Added environment variables:
 - Maven home & PATH
 - Sonar project key/name
 - Nexus URL, repository, version, protocol
 - Tomcat URL
 - Slack channel
- Saved configuration
- Added **Checkout Code** stage (GitHub repo + branch)
- Added **SonarQube Analysis** stage using withSonarQubeEnv
- Added **Maven Build** stage:
 - mvn clean package -DskipTests
- Ensured stages order is correct
- Added **Nexus Artifactory** stage
- Captured WAR from target/*.war
- Configured:
 - Nexus protocol, URL, repository
 - Credentials ID
 - Group ID, artifact ID, version (SNAPSHOT)
- Enabled artifact upload

Added **Deploy on Tomcat** stage

Configured:

- Tomcat credentials
- \${TOMCAT_URL}
- Context path
- WAR file path

Added **post** section:

- Slack notification on **success**
- Slack notification on **failure**
- Pipeline started successfully
- Multiple stages failed (red)
- Indicates configuration issues (branch, repo, credentials)

- Updated **Checkout Code** stage
- Corrected branch name (e.g., main)
- Saved pipeline
- Checkout succeeded
- Some stages still failing (Nexus/Tomcat)
- SonarQube stage passes
- Remove http in front of nexus url
- Logged into **Sonatype Nexus Repository**
- Created/verified **maven2 (hosted)** snapshot repo
- Set:
 - Version policy: Snapshot
 - Layout: Permissive
 - Deployment policy: Allow redeploy

Upload to Nexus (SNAPSHOT) stage succeeds

Artifact visible in Nexus

Confirms Nexus credentials and repo settings are correct

Code of Declarative pipeline job:

```
pipeline {
    agent {
        label 'master'
    }

    environment {
        NEXUS_URL = 'http://54.236.32.112:8081'
        TOMCAT_URL = 'http://54.236.32.112:8080'
    }

    stages {
```

```

stage('Git Clone') {
steps {
    echo "Cloning main branch..."

    git(
        branch: 'main',
        url: 'https://github.com/betawins/hiring-app.git'
    )

    echo "Git clone completed successfully."
}
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv('SonarQube') {
            sh '''
                mvn clean verify \
                org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
                -Dsonar.projectKey=SimpleCustomerApp \
                -Dsonar.projectName=SimpleCustomerApp
            '''
        }
    }
}

stage('Maven Compilation') {
    steps {
        sh '''
            mvn clean package -DskipTests
            ls -lh target/*.war
        '''
    }
}

```

```
}  
}
```

```
stage('Nexus Upload') {
```

```
  steps {
```

```
    withCredentials([
```

```
      usernamePassword(
```

```
        credentialsId: 'nexus-credentials',
```

```
        usernameVariable: 'NEXUS_USER',
```

```
        passwordVariable: 'NEXUS_PASSWORD'
```

```
      )
```

```
    ]){
```

```
      sh '''
```

```
        WAR_FILE=$(ls target/*.war)
```

```
  
        curl -v \
```

```
        -u "$NEXUS_USER:$NEXUS_PASSWORD" \
```

```
        --upload-file "$WAR_FILE" \
```

```
        "$NEXUS_URL/repository/simpleCustomerApp/${basename "$WAR_FILE"}"
```

```
      '''
```

```
    }
```

```
  }
```

```
}
```

```
stage('Slack Notification') {
```

```
  steps {
```

```
    slackSend(
```

```
      channel: '#build_notification',
```

```
      message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Build and Nexus upload  
completed."
```

```
    )
```

```

    }
}

stage('Deploy On Tomcat') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'tomcat_credentials',
                usernameVariable: 'TOMCAT_USER',
                passwordVariable: 'TOMCAT_PASSWORD'
            )
        ]) {
            sh '''
                WAR_FILE=$(ls target/*.war)

                echo "Deploying $WAR_FILE to Tomcat..."

                curl -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
                --upload-file "$WAR_FILE" \
                "$TOMCAT_URL/manager/text/deploy?path=/SimpleCustomerApp&update=true"

                echo "Deployment completed."
            '''
        }
    }
}

post {
    success {
        slackSend(

```

```

        channel: '#build_notification',

        message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Application deployed to
Tomcat successfully."
    )
}

failure {
    slackSend(
        channel: '#build_notification',

        message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Pipeline failed. Check
Console Output."
    )
}
}
}

```

Jenkins / hiring_app_declarative

Changes
Build Now
Delete Pipeline
Configure
Rename
Full Stage View
SonarQube
Stages
Pipeline Syntax

Stage View

Average stage times: (full run time: ~17s)

	Git Clone	SonarQube Analysis	Maven Compilation	Nexus Upload	Slack Notification	Deploy On Tomcat	Declarative: Post Actions
#5 Aug 20 22:09 No Changes	631ms	11s	3s	398ms	135ms	369ms	128ms
#6 Aug 20 22:08 No Changes	616ms	11s	3s	365ms	148ms	373ms	139ms
#7 Aug 20 21:57 No Changes	357ms	13s	3s	431ms	145ms	375ms	137ms
#8 Aug 20 21:54 No Changes	247ms	61ms	50ms	51ms	49ms	50ms	155ms

Permalinks

Builds > Filter /

August 20, 2026

- #9 4:39 PM
- #8 4:38 PM
- #7 4:27 PM

```
+ echo 'Deploying target/hiring.war to Tomcat...'
Deploying target/hiring.war to Tomcat...
+ curl -u *****:**** --upload-file target/hiring.war 'http://18.234.76.11:8080/manager/text/deploy?path=/SimpleCustomerApp&update=true'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload  Total   Spent    Left     Speed

  0   0   0    0   0   0    0   0  --:--:-- --:--:-- --:--:--    0
100 1761   0    63 100 1698 2670 71988 --:--:-- --:--:-- --:--:-- 76565
OK - Deployed application at context path [/SimpleCustomerApp]
+ echo 'Deployment completed.'
Deployment completed.
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions) (hide)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: hari, channel: #build_notification, color: <empty>, botUser:
true, tokenCredentialId: slack_notification_token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

sonatype nexus repository Community Edition

Search components or CVEs...

Browse / simpleCustomerApp

HTML View

Advanced search...

- hiring.war
- SimpleCustomerApp-10-SNAPSHOT.war
- SimpleCustomerApp-11-SNAPSHOT.war
- SimpleCustomerApp-12-SNAPSHOT.war
- SimpleCustomerApp-18-SNAPSHOT.war
- SimpleCustomerApp-20-SNAPSHOT.war
- SimpleCustomerApp-21-SNAPSHOT.war
- SimpleCustomerApp-22-SNAPSHOT.war
- SimpleCustomerApp-8-SNAPSHOT.war

Hari

An offer awaits

Find a conversation...

Huddles

Directories

Activity

Starred

Drag and drop important stuff here

Channels

- # all-hari
- # build_notification
- # new-channel
- # social

+ Add channels

Direct messages

build_notification

Invite teammates

Messages

ADD CANVAS

BUILD FAILED

Application : hiringapp
Branch : main
Build : #5

Check Jenkins console output for details.

Jenkins APP 9:54 PM

FAILED: hiring_app_declarative #6 - Pipeline failed. Check Console Output.

SUCCESS: hiring_app_declarative #7 - Build and Nexus upload completed.

SUCCESS: hiring_app_declarative #7 - Application deployed to Tomcat successfully.

Jenkins APP 10:08 PM

SUCCESS: hiring_app_declarative #8 - Build and Nexus upload completed.

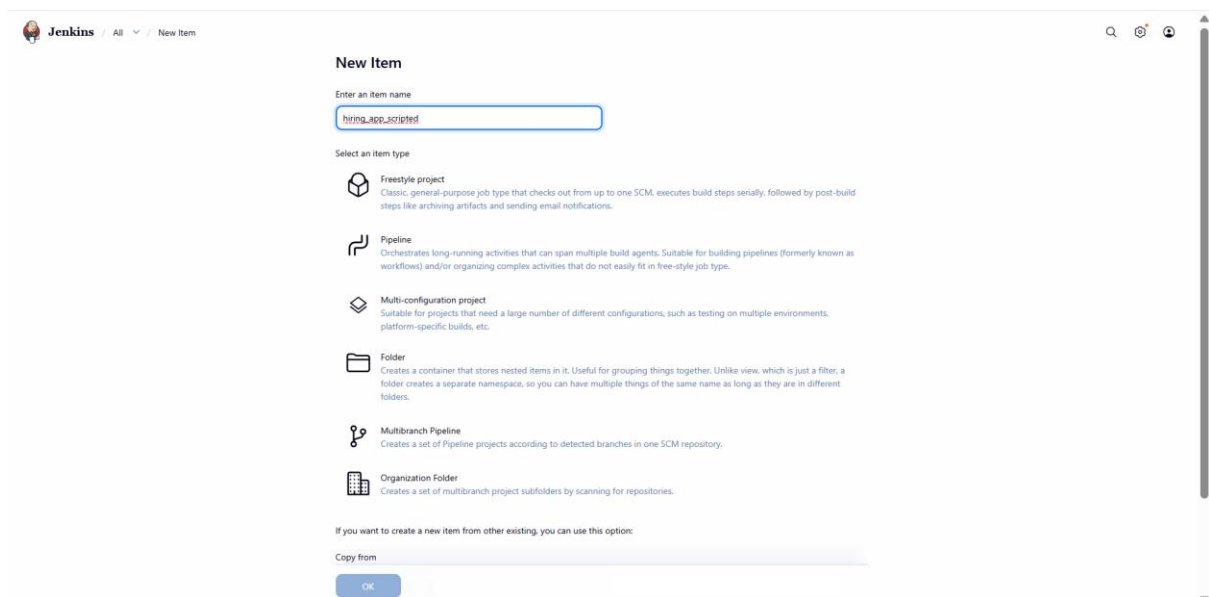
10:08 SUCCESS: hiring_app_declarative #8 - Application deployed to Tomcat successfully.

SUCCESS: hiring_app_declarative #9 - Build and Nexus upload completed.

SUCCESS: hiring_app_declarative #9 - Application deployed to Tomcat successfully.

2.Create one Scripted pipeline job

- Opened **Jenkins** → **New Item**
- Entered job name: **hiring_app_scripted**
- Selected **Pipeline**
- Clicked **OK**



- Opened **Configure** → **Pipeline**
- Used **Scripted Pipeline** (node {})
- Defined environment variables:
 - Maven home and PATH
 - Sonar project key and name
 - Nexus URL, repository, protocol
 - Group ID, artifact ID, version
 - Tomcat URL
 - Slack channel
- Enabled timestamps
- Added **Notify Build Started** Slack stage

- Added **Checkout Code** stage
 - GitHub repository
 - Branch: main
- Added **SonarQube Analysis** stage
 - Used withSonarQubeEnv
 - Maven Sonar scan command
- Added **Maven Build** stage
 - mvn clean package -DskipTests
 - WAR generated in target/
- Added **Upload to Nexus (SNAPSHOT)** stage
- Used nexusArtifactUploader
- Configured:
 - Nexus URL and protocol
 - Snapshot repository
 - Credentials ID
 - Group ID, artifact ID
 - Version: SNAPSHOT
 - WAR file path
- Added **Deploy on Tomcat** stage
- Used Tomcat deploy adapter
- Configured:
 - Tomcat credentials
 - Tomcat URL
 - Context path: hiring
 - WAR file: target/hiring.war
- Added **Notify Success** stage
- Slack message includes:
 - Job name
 - Build number
 - Build URL
- Message color set to **green**

Wrapped pipeline inside try-catch

Added **Notify Failure** Slack stage

Message color set to **danger**

Re-throws error to fail the build properly

Pipeline stages executed successfully:

- Notify Build Started
- Checkout Code
- SonarQube Analysis
- Maven Build
- Upload to Nexus (SNAPSHOT)
- Deploy on Tomcat
- Notify Success

All stages shown in **green**

Full code Scripted pipeline job:

```
node('master') {  
    withEnv([  
        'NEXUS_URL=http://54.236.32.112:8081',  
        'TOMCAT_URL=http://54.236.32.112:8080'  
    ]) {  
  
        try {  
  
            stage('Git Clone') {  
                echo "Cloning main branch..."  
  
                git(  
                    branch: 'main',  
                    url: 'https://github.com/betawins/hiring-app.git'  
                )  
            }  
        }  
    }  
}
```

```

    echo "Git clone completed successfully."
}

stage('SonarQube Analysis') {
    withSonarQubeEnv('SonarQube') {
        sh '''
            mvn clean verify \
            org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
            -Dsonar.projectKey=SimpleCustomerApp \
            -Dsonar.projectName=SimpleCustomerApp
        '''
    }
}

```

```

stage('Maven Compilation') {
    sh '''
        mvn clean package -DskipTests
        ls -lh target/*.war
    '''
}

```

```

stage('Nexus Upload') {
    withCredentials([
        usernamePassword(
            credentialsId: 'nexus-credentials',
            usernameVariable: 'NEXUS_USER',
            passwordVariable: 'NEXUS_PASSWORD'
        )
    ]) {
        sh '''

```

```

WAR_FILE=$(ls target/*.war)

echo "WAR file: $WAR_FILE"
echo "Uploading WAR to Nexus..."

curl -v \
-u "$NEXUS_USER:$NEXUS_PASSWORD" \
--upload-file "$WAR_FILE" \
"$NEXUS_URL/repository/simpleCustomerApp/${basename "$WAR_FILE"}"

echo "Nexus upload completed successfully."
'''
}
}

stage('Deploy On Tomcat') {
    withCredentials([
        usernamePassword(
            credentialsId: 'tomcat_credentials',
            usernameVariable: 'TOMCAT_USER',
            passwordVariable: 'TOMCAT_PASSWORD'
        )
    ]) {
        sh '''
            WAR_FILE=$(ls target/*.war)

            echo "Deploying $WAR_FILE to Tomcat..."

            curl -v \
            -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
            --upload-file "$WAR_FILE" \

```

```

"$TOMCAT_URL/manager/text/deploy?path=/SimpleCustomerApp&update=true"

echo "Deployment completed successfully."

'''

}

}

stage('Slack Notification') {
    slackSend(
        channel: '#build_notification',
        message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Build, Nexus upload
and Tomcat deployment completed successfully."
    )
}

} catch (Exception e) {

    slackSend(
        channel: '#build_notification',
        message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Pipeline failed. Check
Console Output."
    )

    throw e
}
}
}

```

Jenkins / hiring_app_scripted

Status

Changes

Build Now

Delete Pipeline

Configure

Rename

Full Stage View

SonarQube

Stages

Pipeline Syntax

hiring_app_scripted

Add description

Stage View

	Git Clone	SonarQube Analysis	Maven Compilation	Nexus Upload	Deploy On Tomcat	Slack Notification
Average stage times: (full run time: ~15s)	528ms	15s	2s	389ms	373ms	135ms
#0 Aug 21 13:00 No Changes	240ms	10s	2s	360ms	373ms	135ms
#2 Aug 21 12:46 No Changes	682ms	13s	2s	419ms failed		
#1 Aug 21 12:33 No Changes	662ms	21s failed				

Builds >

Filter

Jenkins / hiring_app_scripted

#3

```
< Transfer-Encoding: chunked
< Date: Fri, 21 Aug 2026 07:31:05 GMT
<
{ [74 bytes data]

100 1761 0 63 100 1698 1310 35317 --:--:-- --:--:-- --:--:-- 36687
* Connection #0 to host 54.236.32.112:8080 left intact
OK - Deployed application at context path [/SimpleCustomerApp]
+ echo 'Deployment completed successfully.'
Deployment completed successfully.
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Slack Notification)
[Pipeline] slackSend
Slack Send Pipeline step running, values are - baseUrl:<empty>, teamDomain: hari, channel: #build_notification, color: <empty>, botu
true, tokenCredentialId: slack_notification_token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Files

Channels

all-hari

build_notification

new-channel

social

+ Add channels

Direct messages

srihari48326 you

+ Invite people

Agents & apps

Jenkins

Slack

Today

Jenkins APP 12:34 PM

FAILED: hiring_app_scripted #1 - Pipeline failed. Check Console Output.

Jenkins APP 12:46 PM

FAILED: hiring_app_scripted #2 - Pipeline failed. Check Console Output.

Jenkins APP 1:01 PM

SUCCESS: hiring_app_scripted #3 - Build, Nexus upload and Tomcat deployment completed successfully.

Message #build_notification

+ Aa @

Not secure

54.236.32.112:8080/SimpleCustomerApp/

Welcome to Techie Horizon

3. Create one multi stage pipeline job

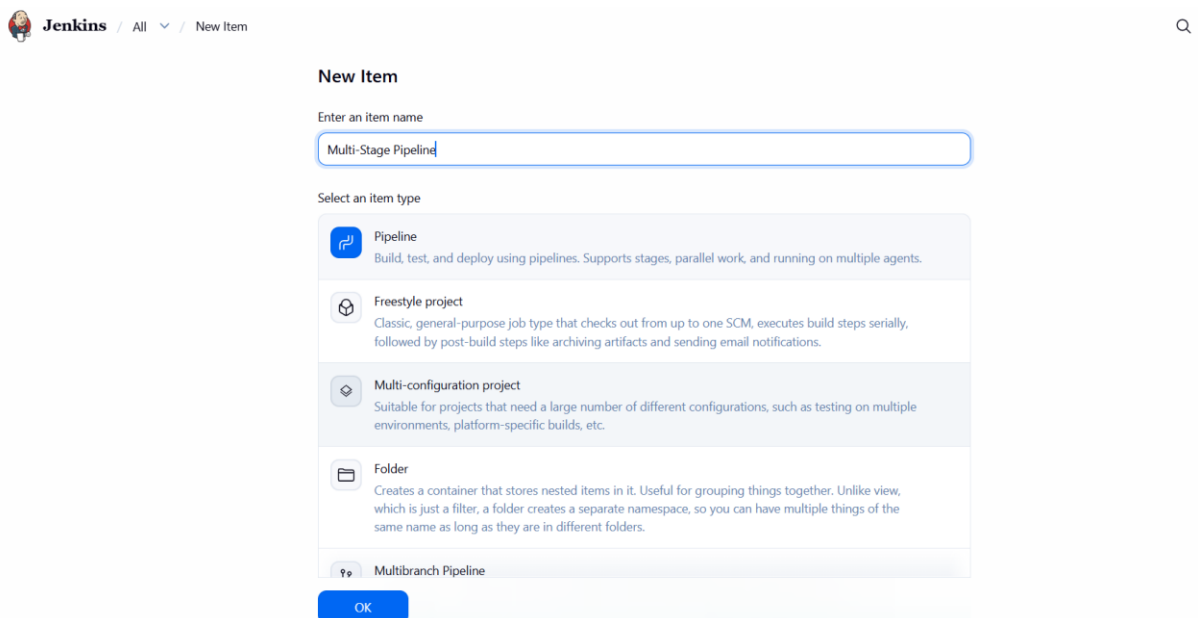
create a **Declarative Multi-Stage Pipeline** using the same application flow:

Git Clone → SonarQube → Maven Build → Nexus Upload → Tomcat Deploy → Slack Notification

1. Create a new Jenkins Pipeline job

In Jenkins:

1. Click **New Item**
2. Enter a name, for example:
hiring_app_multistage
3. Select **Pipeline**
4. Click **OK**
5. Scroll to **Pipeline**
6. Select **Pipeline script**
7. Paste the following Jenkinsfile
8. Click **Save**
9. Click **Build Now**



- Opened **Configure → Pipeline**
- Used **Declarative pipeline**
- Defined:

- agent any
 - timestamps()
- Configured environment variables:
 - Maven home and PATH
 - Sonar project key and name
 - Nexus URL, repository, protocol
 - Group ID, artifact ID, version
 - Tomcat URL
 - Slack channel
- Added **Initialize** stage
- Added **Notify Build Started** stage (Slack)
- Added **Checkout Source Code** stage
 - GitHub repository
 - Branch: main
- Added **Code Quality Analysis** stage
 - Integrated **SonarQube**
 - Executed Maven Sonar scan
- Added **Build Application** stage
 - mvn clean package -DskipTests
 - WAR generated under target/
- Pipeline executed successfully
- Stages completed in order:
 - Initialize
 - Notify Build Started
 - Checkout Source Code
 - Code Quality Analysis
 - Build Application
 - Publish Artifact to Nexus
 - Deploy to Tomcat
 - Post Actions
- All stages shown in **green**

Multi stage pipeline job Code:

```
pipeline {

    agent {

        label 'master'

    }

    environment {

        NEXUS_URL = 'http://54.236.32.112:8081'

        TOMCAT_URL = 'http://54.236.32.112:8080'

    }

    stages {

        stage('Git Clone') {

            steps {

                echo "Cloning main branch..."

                git(

                    branch: 'main',

                    url: 'https://github.com/betawins/hiring-app.git'

                )

                echo "Git clone completed successfully."

            }

        }

        stage('SonarQube Analysis') {

            steps {

                withSonarQubeEnv('SonarQube') {

                    sh ""
```

```

        mvn clean verify \
        org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
        -Dsonar.projectKey=SimpleCustomerApp \
        -Dsonar.projectName=SimpleCustomerApp
    ""
}
}
}

```

```

stage('Maven Compilation') {
    steps {
        sh ""
        mvn clean package -DskipTests
        ls -lh target/*.war
    ""
    }
}

```

```

stage('Nexus Upload') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'nexus-credentials',
                usernameVariable: 'NEXUS_USER',
                passwordVariable: 'NEXUS_PASSWORD'
            )
        ]) {
            sh ""
            WAR_FILE=$(ls target/*.war)

            echo "WAR file: $WAR_FILE"
        }
    }
}

```

```

        echo "Uploading WAR to Nexus..."

        curl -v \
        -u "$NEXUS_USER:$NEXUS_PASSWORD" \
        --upload-file "$WAR_FILE" \
        "$NEXUS_URL/repository/simpleCustomerApp/${basename "$WAR_FILE"}"

        echo "Nexus upload completed successfully."
    ""
}
}
}

stage('Deploy On Tomcat') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'tomcat_credentials',
                usernameVariable: 'TOMCAT_USER',
                passwordVariable: 'TOMCAT_PASSWORD'
            )
        ]) {
            sh ""

            WAR_FILE=$(ls target/*.war)

            echo "Deploying $WAR_FILE to Tomcat..."

            curl -v \
            -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
            --upload-file "$WAR_FILE" \
            "$TOMCAT_URL/manager/text/deploy?path=/SimpleCustomerApp&update=true"
        }
    }
}

```

```

        echo "Deployment completed successfully."
    ""
}
}
}

stage('Slack Notification') {
    steps {
        slackSend(
            channel: '#build_notification',
            message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Build, Nexus upload
and Tomcat deployment completed successfully."
        )
    }
}

post {

    failure {
        slackSend(
            channel: '#build_notification',
            message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Pipeline failed. Check
Console Output."
        )
    }

    success {
        echo "Pipeline completed successfully."
    }
}

```

}

 Jenkins

Multi-Stage Pipeline

🔍

⚙️

👤

Status

</> Changes

▶ Build Now

🗑 Delete Pipeline

⚙️ Configure

✎ Rename

🔍 Full Stage View

🔊 SonarQube

📁 Stages

❓ Pipeline Syntax

Builds >

🔍 Filter

/

Today

🟢 #1 7:48 AM

🔊

⌵

Multi-Stage Pipeline

Add description

Stage View

Average stage times:
(full run time: ~16s)

	Git Clone	SonarQube Analysis	Maven Compilation	Nexus Upload	Deploy On Tomcat	Slack Notification	Declarative: Post Actions
#1 Aug 21 13:18 No Changes	388ms	10s	2s	374ms	383ms	169ms	64ms

SonarQube Quality Gate

SimpleCustomerApp **Passed**
server-side processing: **Success**

Permalinks

- Last build (#1), 7 min 42 sec ago
- Last stable build (#1), 7 min 42 sec ago
- Last successful build (#1), 7 min 42 sec ago
- Last completed build (#1), 7 min 42 sec ago

all-hari

build_notification

new-channel

Jenkins

APP 1:18 PM

SUCCESS: Multi-Stage Pipeline #1 - Build, Nexus upload and Tomcat deployment completed successfully.

4. Create one parallel stage pipeline job

- Open Jenkins dashboard
- Click **New Item**
- Enter job name: **parallel_stage**
- Select **Pipeline**
- Click **OK**

Jenkins / All / New Item

New Item

Enter an item name

Parallel Stage Pipeline

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline

OK

Go to **Configure** → **Pipeline**

Set **Definition** to **Pipeline script**

Paste the pipeline script that contains:

- pipeline { agent any }
- environment {} block for Maven, SonarQube, Nexus, Tomcat, Slack
- Multiple stage {} blocks
- One stage using **parallel { }**

Parallel stage pipeline job Code:

```
pipeline {  
  
    agent {  
        label 'master'  
    }  
  
    environment {  
        NEXUS_URL = 'http://54.236.32.112:8081'  
        TOMCAT_URL = 'http://54.236.32.112:8080'  
    }  
  
    stages {  
  
        stage('Git Clone') {  
            steps {  
                echo "Cloning main branch..."  
  
                git(  
                    branch: 'main',  
                    url: 'https://github.com/betawins/hiring-app.git'  
                )  
  
                echo "Git clone completed successfully."  
            }  
        }  
  
        stage('Parallel Checks') {  
  
            parallel {
```

```
stage('SonarQube Analysis') {  
    steps {  
        withSonarQubeEnv('SonarQube') {  
            sh '''  
                mvn clean verify \  
                org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \  
                -Dsonar.projectKey=SimpleCustomerApp \  
                -Dsonar.projectName=SimpleCustomerApp  
            '''  
        }  
  
        echo "SonarQube analysis completed."  
    }  
}
```

```
stage('Maven Build') {  
    steps {  
        sh '''  
            mvn clean package -DskipTests  
            ls -lh target/*.war  
        '''  
  
        echo "Maven build completed."  
    }  
}
```

```
stage('System Information') {  
    steps {  
        sh '''  
            echo "Checking Jenkins server..."
```

```
echo "Hostname:"
```

```
hostname
```

```
echo "Java Version:"
```

```
java -version
```

```
echo "Maven Version:"
```

```
mvn -version
```

```
echo "Disk Usage:"
```

```
df -h
```

```
'''
```

```
echo "System information check completed."
```

```
}
```

```
}
```

```
}
```

```
}
```

```
stage('Nexus Upload') {
```

```
  steps {
```

```
    withCredentials([
```

```
      usernamePassword(
```

```
        credentialsId: 'nexus-credentials',
```

```
        usernameVariable: 'NEXUS_USER',
```

```
        passwordVariable: 'NEXUS_PASSWORD'
```

```
      )
```

```
    ]){
```

```
      sh '''
```

```
        WAR_FILE=$(ls target/*.war)
```

```

        echo "WAR file: $WAR_FILE"

        echo "Uploading WAR to Nexus..."

        curl -v \
        -u "$NEXUS_USER:$NEXUS_PASSWORD" \
        --upload-file "$WAR_FILE" \
        "$NEXUS_URL/repository/simpleCustomerApp/${basename "$WAR_FILE"}"

        echo "Nexus upload completed successfully."
    ""
}
}
}

stage('Deploy On Tomcat') {
    steps {
        withCredentials([
            usernamePassword(
                credentialsId: 'tomcat_credentials',
                usernameVariable: 'TOMCAT_USER',
                passwordVariable: 'TOMCAT_PASSWORD'
            )
        ]) {
            sh ""

            WAR_FILE=$(ls target/*.war)

            echo "Deploying $WAR_FILE to Tomcat..."

            curl -v \
            -u "$TOMCAT_USER:$TOMCAT_PASSWORD" \
            --upload-file "$WAR_FILE" \

```

```
"$TOMCAT_URL/manager/text/deploy?path=/SimpleCustomerApp&update=true"
```

```
echo "Deployment completed successfully."
```

```
'''
```

```
}
```

```
}
```

```
}
```

```
}
```

```
post {
```

```
    success {
```

```
        slackSend(
```

```
            channel: '#build_notification',
```

```
            message: "SUCCESS: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Parallel checks, Nexus  
upload and Tomcat deployment completed successfully."
```

```
        )
```

```
    }
```

```
    failure {
```

```
        slackSend(
```

```
            channel: '#build_notification',
```

```
            message: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER} - Parallel pipeline failed.  
Check Console Output."
```

```
        )
```

```
    }
```

```
}
```

```
}
```

Maven home path configured

SonarQube project key and name defined

Nexus URL, repository, version, protocol defined

Tomcat URL defined

Slack channel defined

Timestamps enabled in options

- Stage name: **Initialize**
- Prints initialization message using echo
- Stage name: **Checkout Source Code**
- Git repository URL configured
- Branch set to **main**

Stage name: **Parallel Tasks**

Inside this stage, parallel {} block used

SonarQube analysis runs as one parallel task

Maven build runs in parallel

- Uses withSonarQubeEnv
 - Runs Maven verify
 - Executes Sonar scanner with project key and name
 - Quality gate result shown as **Passed**
-
- Runs Maven clean package
 - Skips tests using -DskipTests
 - WAR file generated in target/

Stage name: **Deploy to Tomcat**

Uses Tomcat deployment adapter

Credentials configured

Context path set

WAR deployed successfully

On **success**

- Slack success notification sent

On failure

- Slack failure notification sent
- Click **Build Now**
- Stage View shows:
 - Parallel execution happening
 - Green blocks for success
- Total build time reduced due to parallel execution
- Pipeline completed successfully
- Parallel stage executed correctly
- Artifact uploaded to Nexus
- Application deployed to Tomcat
- Slack notifications triggered

The screenshot displays the Jenkins interface for a pipeline named "Parallel Stage Pipeline". On the left, a sidebar contains navigation links: Status, Changes, Build Now, Delete Pipeline, Configure, Rename, Full Stage View, Stages, and Pipeline Syntax. The main area shows the "Stage View" with a table of stage execution times. Above the table, it states "Average stage times: (full run time: ~15s)". The table has columns for various stages: Git Clone, Parallel Checks, SonarQube Analysis, Maven Build, System Information, Nexus Upload, Deploy On Tomcat, and Declarative: Post Actions. Each column shows the time taken for that stage in a specific build. Below the table, there is a "Permalinks" section. At the bottom, a Slack notification is visible, stating "SUCCESS: Parallel Stage Pipeline #1 - Parallel checks, Nexus upload and Tomcat deployment completed successfully."

	Git Clone	Parallel Checks	SonarQube Analysis	Maven Build	System Information	Nexus Upload	Deploy On Tomcat	Declarative: Post Actions
Average stage times: (full run time: ~15s)	379ms	108ms	13s	5s	1s	380ms	369ms	160ms
#1 Aug 21 13:31 No Changes	379ms	108ms	13s	5s	1s	380ms	369ms	160ms

Permalinks

Slack Notification: SUCCESS: Parallel Stage Pipeline #1 - Parallel checks, Nexus upload and Tomcat deployment completed successfully.



Jenkins

Parallel Stage Pipeline

#1

Stages

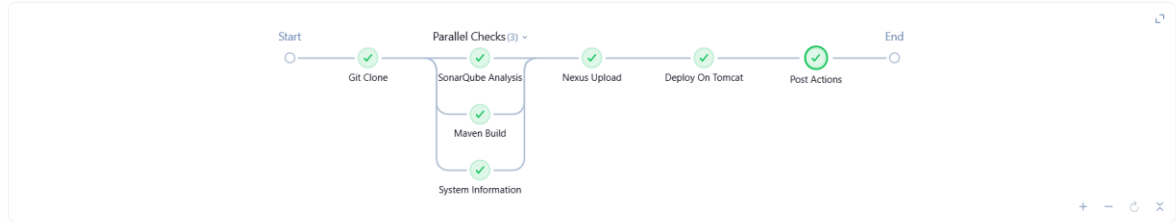


✓ #1

Rerun



Started by BOORA SRIHARI Started 2 min 56 sec ago Took 15 sec



Search

- ✓ Git Clone 0.35s
- ✓ Parallel Checks 13s
- ✓ Nexus Upload 0.36s
- ✓ Deploy On Tomcat 0.35s
- ✓ Post Actions 0.14s

✓ Post Actions 0.14s Started 3m ago Jenkins

✓ Send Slack Message 0.11s

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: hari, channel: #build_notification, color: <empty>, botUser: true, tokenCredentialId: slack_notification_token, notifyCommitters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

